Hello World!
UVA 11636

Explanation By Dr. Kobti

(with a less than perfect solution)

The Problem

When you first made the computer to print the sentence “Hello World!”, you felt so happy, not knowing how complex and interesting the world of programming and algorithm will turn out to be. Then you did not know anything about loops, so to print 7 lines of “Hello World!”, you just had to copy and paste some lines. If you were intelligent enough, you could make a code that prints “Hello World!” 7 times, using just 3 paste commands. Note that we are not interested about the number of copy commands required. A simple program that prints “Hello World!” is shown in Figure 1. By copying the single print statement and pasting it we get a program that prints two “Hello World!” lines. Then copying these two print statements and pasting them, we get a program that prints four “Hello World!” lines. Then copying three of these four statements and pasting them we can get a program that prints seven “Hello World!” lines (Figure 4). So three pastes commands are needed in total and Of course you are not allowed to delete any line after pasting. Given the number of “Hello World!” lines you need to print, you will have to find out the minimum number of pastes required to make that program from the origin program shown in Figure 1.

The Problem: key sentences

When you first made the computer to print the sentence “Hello World!”, you felt so happy, not knowing how complex and interesting the world of programming and algorithm will turn out to be. Then you did not know anything about loops, so to print 7 lines of “Hello World!”, you just had to copy and paste some lines. If you were intelligent enough, you could make a code that prints “Hello World!” 7 times, using just 3 paste commands. Note that **we are not interested about the number of copy commands required**. A simple program that prints “Hello World!” is shown in Figure 1. By copying the single print statement and pasting it we get a program that prints two “Hello World!” lines. Then copying these two print statements and pasting them, we get a program that prints four “Hello World!” lines. Then copying three of these four statements and pasting them we can get a program that prints seven “Hello World!” lines (Figure 4). So three pastes commands are needed in total and Of course you are not allowed to delete any line after pasting. Given the number of “Hello World!” lines you need to print, you will have to **find out the minimum number of pastes required** to make that program from the origin program shown in Figure 1.

Figure 1

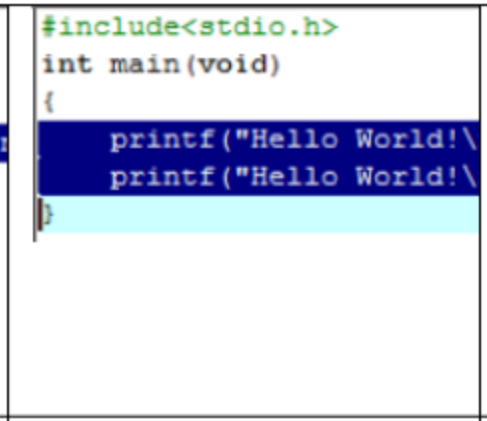

Figure 2

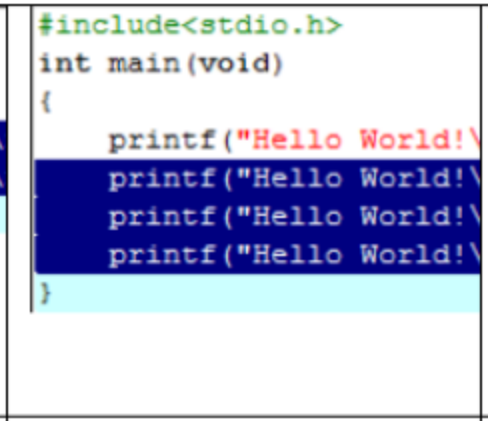

Figure3

Figure 4

The Problem

- **Input**
- The input file can contain up to 2000 lines of inputs. Each line contains an integer N (0 < N < 10001) that denotes the number of “Hello World!” lines are required to be printed. Input is terminated by a line containing a negative integer.
- **Output**
- For each line of input except the last one, produce one line of output of the form ‘Case X: Y ’ where X is the serial of output and Y denotes the minimum number of paste commands required to make a program that prints N lines of “Hello World!”.
- **Sample Input**

  2

  10

  -1
- **Sample Output**

  Case 1: 1

  Case 2: 4

The Problem

- **Input**
- The input file can contain **up to 2000 lines of inputs**. Each line contains an integer **N (0 < N < 10001)** that denotes the number of "Hello World!" lines are required to be printed. Input is **terminated by a line containing a negative integer**.
- **Output**
- For each line of input except the last one, produce one line of output of the form '**Case X: Y** ' where X is the serial of output and Y denotes the minimum number of paste commands required to make a program that prints N lines of "Hello World!".
- **Sample Input**

  2 ← 0 < N < 10001

  10 can have up to 2000 input cases

  -1 ← any negative number
- **Sample Output**

  Case 1: 1

  Case 2: 4

Step 1: Analysis (assumptions and objectives)

Read the question – refer to the notes in red

Step 2: Design
(How to solve)

Proceed to this step only if you fully understand what the problem is asking for.

Pencil and paper(s) often required

Programming language is not important now, we need to come up with the step by step solution for this problem from input to output

Step 2: Design
(How to solve)

If you always copy all the available lines and paste them you generate this sequence:

1 2 4 8 16 32 64 128 256 512 1024 …

e.g. starting with 1 line, copy and paste it, you will have another line to a total of 2.
Do it again, copy all available lines (2) and paste them, you will get the maximum of 4 lines.
Do it again, copy all available lines (4) and paste them, you will get the maximum of 8 lines.
Hence, if we always copy paste the maximum number of lines we will generate the sequence above.

Step 2: Design (How to solve)

Well… if you want 7 lines, it means that once you have the first line copied into memory, you would: Paste 1 line to get a total of 2 lines, then copy 2 lines, Paste 2 lines to get a total of 4 lines, then copy the remaining lines needed (3 in this case) to get the total of 7 after pasting the 3.

1 2 4 | 8 16 32 64 128 256 512 1024 …

starting with 1 line

2 max copy-pastes

+ 1 remainder paste

= 3 pastes

Step 2: Design (How to solve)

Let's try enough examples until we see a pattern:

Input 1: no pastes needed, so 0 pastes (0)

Input 2: 1 paste needed (1)

Input 3: 1 paste gets the 2, and 1 more to get 3 (3)

Input 4: 2 pastes

Inputs 5 to 7: 2 pastes + 1

Input 8: 3 pastes

Inputs 9 to 15: 3 pastes + 1

Input 16: 4 pastes

sequence: 1 2 4 8 16

Key: 2^0 2^1 2^2 2^3 2^4

Step 2: Design (How to solve)

The pattern!

Sequence (N): 1 2 4 8 16

Key: 2^0 2^1 2^2 2^3 2^4

Given the input 0 < N < 10001

We are looking for x such that:

$2^x = N$

If N is in the sequence, else "round" the input N to be the next one in the sequence and then calculate x

Step 2: Design (How to solve)

The pattern!

Sequence (N): 1 2 4 8 16

Key: 2^0 2^1 2^2 2^3 2^4

Recall that:

$2^x = N$

Solve for x (high school math!):

$\text{Log}_2\ 2^x = \log_2 N$

$x = \log_2 N$

Step 2: Design (How to solve)

The pattern!

Sequence (N): 1 2 4 8 16

Key: 2^0 2^1 2^2 2^3 2^4

Given the in

We are look

$2^x = N$

If N is in the

the input N

sequence and then calculate x

Alternately, instead of changing N, it makes more sense to calculate x then round it up.
Example:
N=10 → x = $\log_2 10$ = 3.32192809488734
Round up:
ceil(3.3219280948873) = 4
Hence, for input 10 the output is 4

Step 2: Design
(How to solve)

Algorithm:

1. **Read N**
2. **Calculate x = ceil($\log_2 N$)**

Test it (refer to sample input):

N=2 → ceil($\log_2 2$) = 1 ☑

N=10 → ceil($\log_2 10$) = 4 ☑

Step 2: Design (Complete Design)

Algorithm allowing us to read multiple cases (up to 2000):

1. Set case_counter = 1
2. Read N
3. While(N > 0)
    1. Set x = ceil($\log_2$N)
    2. Output "Case case_counter++: x"
    3. Read N

Step 3: Implementation (Code using C with test features)

```
#include <stdio.h>
#include <math.h>
/* don't forget "gcc -lm" switch to compile

 use DEBUG 1 if you want view debug steps for testing,
 or 0 if you want production */
#define DEBUG 0

int main()
{
          int case_counter = 1;
          double n;
          double x;

          scanf("%lf", &n);
          if (DEBUG) fprintf(stderr, "first input: [%lf]\n", n);
          while(n > 0)
          {
                    if (DEBUG) fprintf(stderr, "inside loop...\n");
                    x = ceil(log2(n)); /* log is base 2 */
                    if (DEBUG) fprintf(stderr, "x is [%lf]\n", x);
                    printf("Case %d: %d\n", case_counter++, (int)(x));
                    scanf("%lf", &n);
                    if (DEBUG) fprintf(stderr, "input: [%lf]\n", n);
          }
          return 0;
}
```

Compile and test

Compile:

cc -lm hello_world.c

Run and try it:

./a.out

Better to do this way:

./a.out < hello_world.in

```
Create an input text
file containing
exactly what input you
want to type from the
keyboard. Then apply
it using input
redirection "<":

hello_world.in
1
10
-1
```

Step 4: Test thoroughly before submitting

- Create more test cases
- Always test boundary cases, e.g. cases where N is 1 and N is 10000
- Other special cases (as long as they are valid cases)